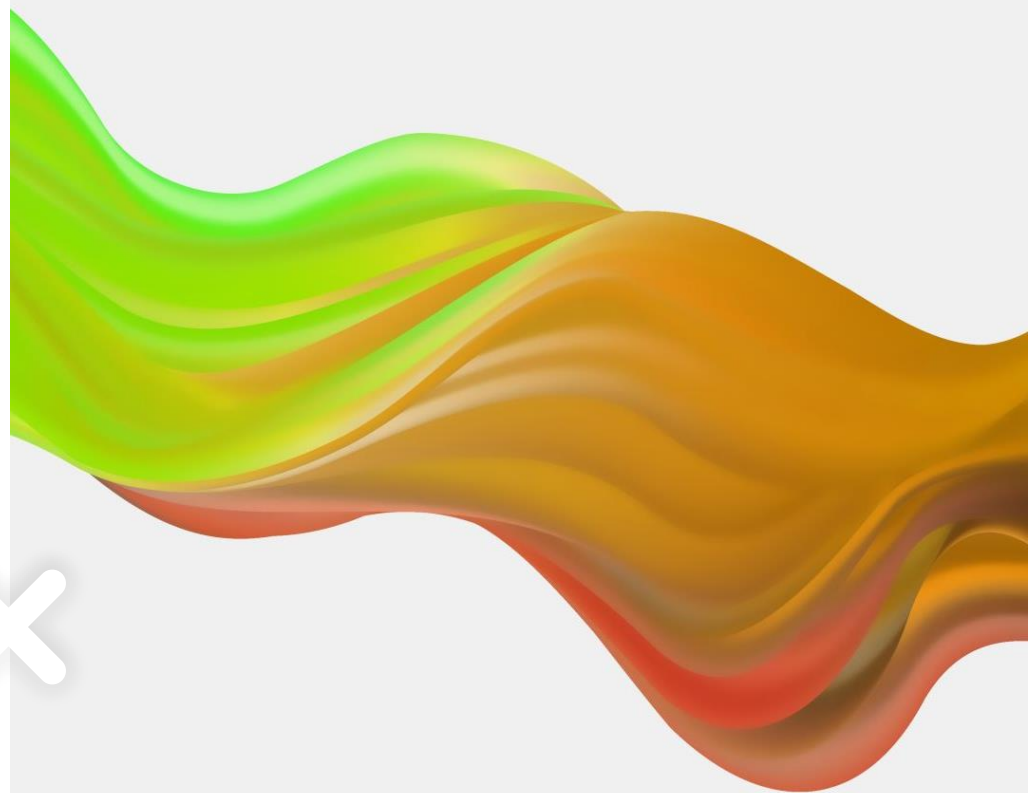




Computer Programming

MENG2020





REVIEW



About our lecture



The if-elseif-else-end Structure

The if-elseif-else-end structure lets you choose one of three (or more) sections of code to execute.

Syntax:

if **cond1**

statement(s) if cond1 true

elseif **cond2**

statement(s) if cond1 false, cond2 true

elseif **cond3**

statement(s) if cond1 false, cond2 false, cond3 true

else

statements(s) if all three conditions false

end

Write a code with **if-elseif-else-end** which asks for **Noise level?**
and assign it to variable **db**.
If **db <= 50** it prints Quiet
db <= 70 Intrusive
db <= 90 Deafening
Otherwise Dangerous

The if-elseif-else-end Structure : Example

Example (script saved as noise.m):

```
db = input('Noise level? ');
if db <= 50
    fprintf('Quiet\n');
elseif db <= 70
    fprintf('Intrusive\n');
elseif db <= 90
    fprintf('Deafening\n');
else
    fprintf('Dangerous\n');
end
```

```
>> noise
Noise level? 56
Intrusive
>> noise
Noise level? 32
Quiet
>> noise
Noise level? 94
Dangerous
>>
```

The switch-case Statement

- The **switch-case** statement is a useful alternative for multiple branches (not just true and false).
- It works not with conditions but with control values (like 1,2,3... or 'A','B','C'... or 'Red', 'Green', 'Blue', 'Yellow' ...).
- The control values must be integer numbers or string scalars. They must be discrete, never ranges.

The switch-case Statement : Structure

```
..... MATLAB program.  
.....  
switch switch expression  
    case value1  
        .....  
        ..... ] Group 1 of commands.  
    case value2  
        .....  
        ..... ] Group 2 of commands.  
    case value3  
        .....  
        ..... ] Group 3 of commands.  
    otherwise  
        .....  
        ..... ] Group 4 of commands.  
end  
..... MATLAB program.  
.....
```

Figure 6-4: The structure of a switch-case statement.



Write a code with switch for traffic light control.

```
ask for light color  
If the selection is R, STOP  
If the selection is Y,CAUTION!  
If the selection is G,GO!  
Invalid color otherwise
```



The switch-case Statement : Example

Here is an example (script saved as light.m):

```
color = input ('Enter light color: ','s');
switch color
    case 'R'
        fprintf ('STOP!\n');
    case 'r'
        fprintf ('STOP!\n');
    case 'Y'
        fprintf ('CAUTION!\n');
    case 'y'
        fprintf ('CAUTION!\n');
    case 'g'
        fprintf ('GO!\n');
    case 'G'
        fprintf ('GO!\n');
    otherwise
        fprintf ('Invalid color\n');
end
```

Repeated to
account for
uppercase and
lowercase
answers
(optional).

```
>> light
Enter light color: r
STOP!
>> light
Enter light color: y
CAUTION!
>> light
Enter light color: B
Invalid color
```

SCRIPT

RESULT

Repetition and Loops

- **Loop:** A control structure that repeats a group of steps (statements) in a program (A special MATLAB repetition command + The commands that will be repeated) .
- **Loop Body:** Only the commands that are repeated in the loop.



Types of Loops

- **Counting Loop:** A loop with a fixed number of repetitions.

Ex: Repeat 100 times.

- **Sentinel-Controlled Loop:** A loop that reads values from a file or keyboard and stops reading when a certain value (called a sentinel) is encountered.

Ex: Read numbers continuously until you encounter a value of -99.

Do I Need Loops?

1. Are there any steps you must repeat to solve the problem? *If yes, you need a loop.*
2. Do you know in advance the number of repetitions? *If yes, you need a counting loop, in MATLAB, a **for-end** loop. If not, you will need a conditional loop, in MATLAB, a **while-end** loop.*
3. Do you know when or how to stop the repetition? *If not, you will not be able to program the loop.*

Types of Loops

- **End-of-File Controlled Loop:** A loop that reads values from a file and stops when there are no more values to read.

Ex: Read numbers continuously until you encounter the end of the file.

- **Input Validation Loop:** A loop that keeps asking a value from the user until the user gets it right.

Ex: Ask the user for a number between 1 and 5 and ask again if the user does not comply.

The for-end Command

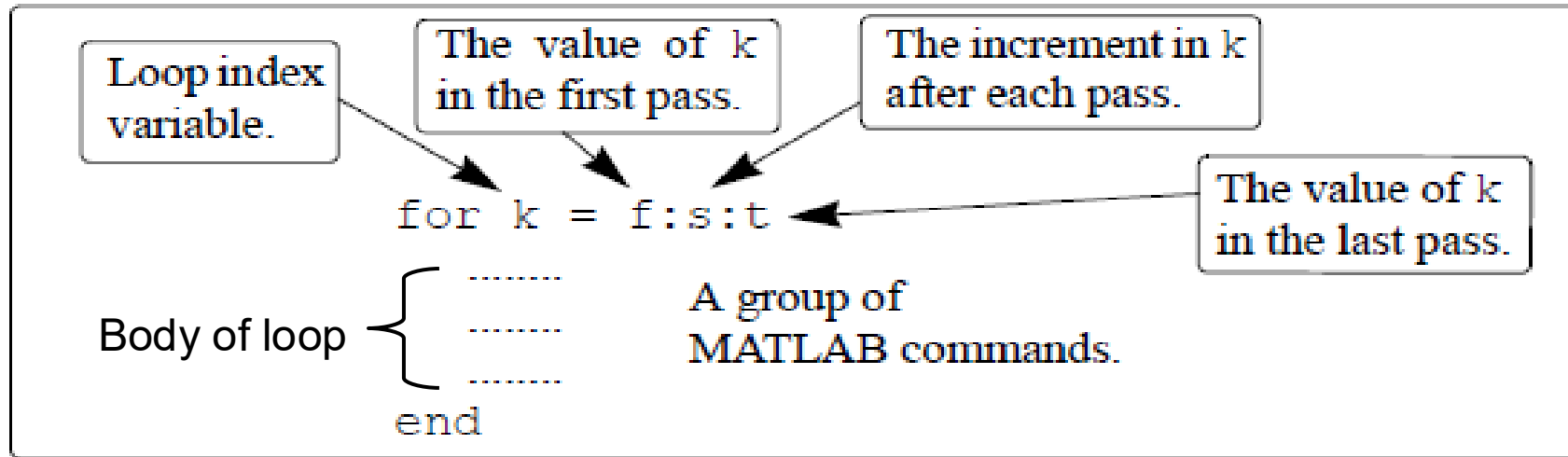


Figure 6-5: The structure of a `for-end` loop.

- The **for-end** format is used for counting loops. The loop index variable can have any variable name (`k`, `m`, or `n` are commonly used).

Note: `i` and `j` are popular choices in other programming languages but should be avoided in MATLAB because they have other uses.

Simple for-end Loop : Examples

1. Printing 100 question marks.
2. Counting by two from 15 to 100.

Simple for-end Loop : Examples

1. Printing 100 question marks.

```
for n = 1:1:100  
    fprintf ('?');  
end
```

```
????????????????????????????????????????????????????????  
????????????????????????????????????????????????????????
```

2. Counting by two from 15 to 100.

```
for n = 15:2:100  
    fprintf ('%d ', n);  
end
```

```
15 17 19 21 23 25 27 29 31 33 35 37 39 41 43 45 47 49  
51 53 55 57 59 61 63 65 67 69 71 73 75 77 79 81 83 85  
87 89 91 93 95 97 99
```

Simple for-end Loop : Examples

3. Counting down from 60 to zero. Then print
LIFTOFF!

Simple for-end Loop : Examples

3. Counting down from 60 to zero.

```
for n = 60:-1:0
    fprintf ('%d ', n);
end

fprintf ('\nLIFTOFF!\n')
```

```
60 59 58 57 56 55 54 53 52 51 50 49 48 47 46 45 44
43 42 41 40 39 38 37 36 35 34 33 32 31 30 29 28 27
26 25 24 23 22 21 20 19 18 17 16 15 14 13 12 11 10
9 8 7 6 5 4 3 2 1 0
LIFTOFF!
```

for-end Loops Based on Vectors

- In the for command `k` can also be assigned a specific value (typed in as a vector).

For example: `for k = [7 -1 3 5]`

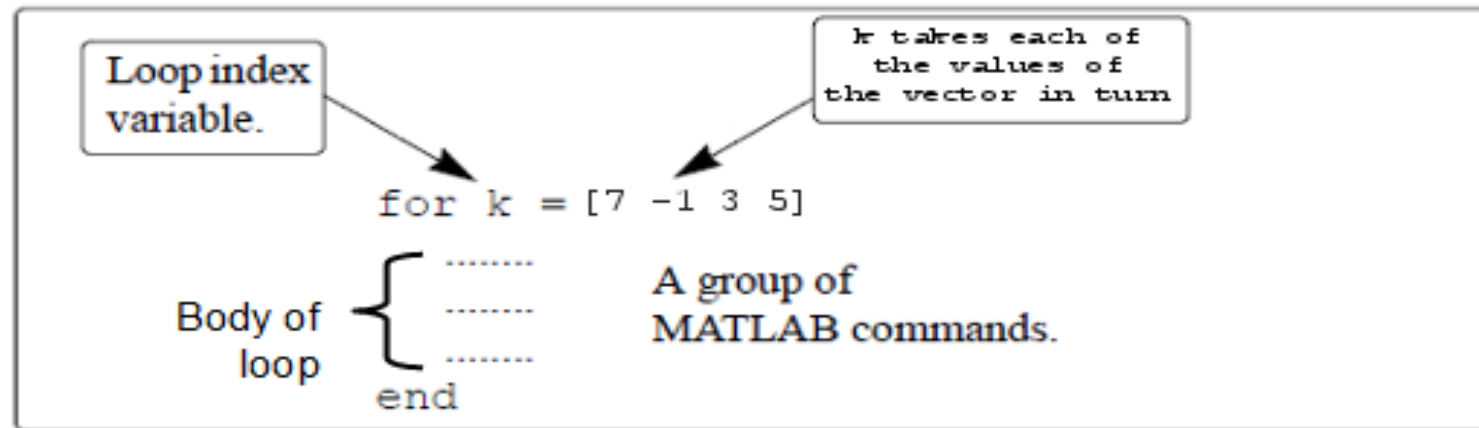


Figure 6-5: The structure of a for-end loop.



You should avoid to change the value of the loop variable (`k` in this example) in the body of the loop unless you really know what you're doing!

for-end Loop Example

Script:

```
for k=1:3:10
```

```
    k
```

```
    x = k^2
```

```
end
```

```
fprintf('After loop k = %d\n', k);
```

for-end Loop Example

Script:

```
for k=1:3:10
```

```
    k
```

```
    x = k^2
```

```
end
```

```
fprintf('After loop k =  
%d\n', k);
```

Output:

```
k = 1
```

```
x = 1
```

```
k = 4
```

```
x = 16
```

```
k = 7
```

```
x = 49
```

```
k = 10
```

```
x = 100
```

```
After loop k = 10
```

Now try *for k=1:3:11* and see what happens!

More for-end Loop Examples

```
for k=1:2:6
k
k^2
end
fprintf('After loop k = %d\n', k);
```



```
for k=1:2:7
k
k^2
end
fprintf('After loop k = %d\n', k);
```



More for-end Loop Examples

```
for k=1:2:6
k
k^2
end
fprintf('After loop k = %d\n', k);
```

.....▶

```
k = 1
ans = 1
k = 3
ans = 9
k = 5
ans = 25
After loop k = 5
```

```
for k=1:2:7
k
k^2
end
fprintf('After loop k = %d\n', k);
```

.....▶

```
k = 1
ans = 1
k = 3
ans = 9
k = 5
ans = 25
k = 7
ans = 49
After loop k = 7
```


for-end Loops vs. Elementwise Operations

You can often calculate something using either a for-loop or elementwise operations.

Elementwise operations are:

- Often faster.
- Often easier to read.
- More MATLAB-like.



TIP

Use elementwise operations when you can, for-loops only when you have to.

for-end Loops vs. Elementwise Operations

Displaying the squares of the numbers between 2 and 10.

`%loop`

`%elementwise`



FRONTIER PROGRAMMER

for-end Loops vs. Elementwise Operations

Displaying the squares of the numbers between 2 and 10.

```
%loop  
for n = 2:1:10  
    fprintf('%d ', n^2);  
end
```

4 9 16 25 36 49 64 81 100



FRONTIER PROGRAMMER

```
%elementwise  
v = 2:1:10  
v.^2
```

4 9 16 25 36 49 64 81 100

**SAME RESULT
BUT FASTER
EXECUTION!**

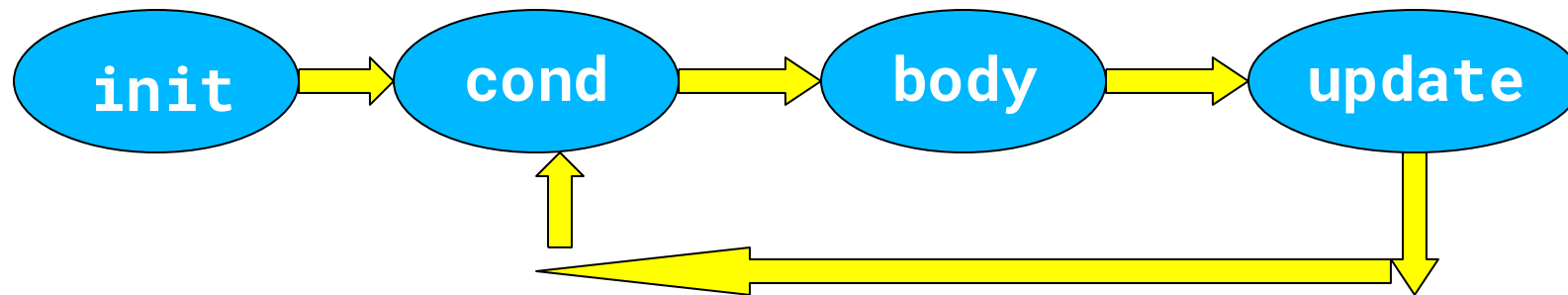
The while-end Loop Statement

- The while-end loop format is used when you don't know in advance the number of loop iterations. It is used for sentinel-controlled loops, end-of-file-controlled loops, and general conditional loops. Here are the steps:

1. *The loop evaluates the conditional expression.*
2. *If the conditional expression is true, it executes the code in the loop body, then goes back to Step 1.*
3. *If the conditional expression is false, it skips the code in the loop body and goes to the code after the end statement.*

The while-end Loop Statement

The **conditional** expression of a while-end loop has a variable in it (called the loop variable). The **body** of the loop *must* change the value of that variable, it is called the **update**. The update must change the value of the variable in a way that the condition *eventually becomes false*. It is important, otherwise, you would be stuck in an *infinite loop**. The loop variable must also be **initialized** before entering the loop.



**If your program gets caught in an infinite loop, put the cursor in the command window and press CTRL+C.*

Basic while-end Loop Examples

1. *A sentinel-controlled loop: Sum all the numbers entered by the user until the sentinel is entered (-99).*

Basic while-end Loop Examples

1. A sentinel-controlled loop: Sum all the numbers entered by the user until the sentinel is entered (-99).

```
sum = 0;
n = input ('Enter an integer number (enter -99 to finish input): ');
while n ~= -99
    sum = sum + n;
    n = input ('Enter an integer number (enter -99 to finish input): ');
end
fprintf ('The sum of the numbers is: %d\n', sum);
```

```
Enter an integer number (enter -99 to finish input): 4
Enter an integer number (enter -99 to finish input): 5
Enter an integer number (enter -99 to finish input): 7
Enter an integer number (enter -99 to finish input): -99
The sum of the numbers is: 16
```

Basic while-end Loop Examples

2. An end-of-file-controlled loop: Sum all the numbers in a file. *feof* returns false if there are still numbers to be extracted from the file.

```
fid = fopen ('data.txt');  
total = 0;  
while ~feof (fid)  
    n = fscanf (fid, '%d', 1);  
    total = total + n;  
end  
fprintf ('The sum of the numbers is: %d\n', total);  
fclose (fid);
```

You need to specify to read one (1) number at a time. If you don't, fscanf will read the entire file into a column vector.

The sum of the numbers is: 166

The sum of all the numbers in the file data.txt.

Basic while-end Loop Examples

3. *An input validation loop. It enforces that the value entered must be a number between 1 and 5.*

Basic while-end Loop Examples

3. *An input validation loop. It enforces that the value entered must be a number between 1 and 5.*

```
n = input ('Enter an integer number between 1 and 5: ');  
  
while n < 1 | n > 5  
    n = input ('This is not a number between 1 and 5! Enter again: ');  
end  
  
fprintf ('Congratulations! You entered: %d\n', n);
```

```
Enter an integer number between 1 and 5: 0  
This is not a number between 1 and 5! Enter again: 7  
This is not a number between 1 and 5! Enter again: 3  
Congratulations! You entered: 3
```

About our lecture

Polynomials, Curve
Fitting, Interpolation
and Numerical
Analysis

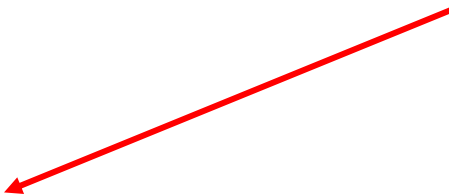


Troubleshooting while-end Loops

★ Common causes of infinite loops:

1. You forgot to change (update) the loop variable value.

```
distance1 = 1;  
distance2 = 10;  
distance3 = 0;  
while distance1 < distance2  
    fprintf('Distance = %d\n', distance3);  
end
```



Infinite loop!
Condition always true!

Troubleshooting while-end Loops

★ Common causes of infinite loops:

2. You update the value of the loop variable but in the **wrong** direction.

```
distance1 = 1;  
distance2 = 10;  
distance3 = 0;  
while distance1 < distance2  
    fprintf('Distance = %d\n', distance3);  
    distance1 = distance1 - 1;  
end
```

Infinite loop!
Condition always
true!

Should be a + here. Remember that the condition
must become false eventually!

Troubleshooting while-end Loops

★ Common causes of infinite loops:

3. You update the value of the wrong variable.

```
distance1 = 1;  
distance2 = 10;  
distance3 = 0;  
while distance1 < distance2  
    fprintf('Distance = %d\n', distance3);  
    distance3 = distance3 + 1;  
end
```

Infinite loop again!
Both variables in the
condition never change
values

distance3 is not a loop variable, it is not
in the condition!

Nested Loops and Nested Selection Statements

If a repetition or selection statement is placed inside another repetition or selection statement, the former are said to be *nested* in the latter.

A most common situation is having an **if-end** statement inside a **for-end** or a **while-end** loop.

Each repetition and selection statement must have an end statement.

```
%prime numbers below 50
for k = 1:1:50
    if isprime(k)
        fprintf ('%d ',
k);
    end
end
```

2 3 5 7 11 13 17 19 23
29 31 37 41 43 47

2 3 5 7 11

Nested Loops

- The inner loop must be completely embedded in the outer loop (no overlaps).
- Each loop must be controlled by a different loop control variable.
 - Suppose we want to display a 6 x 12 multiplication table.
 - We can use the outer loop for the multiplier and the inner loop for the multiplicand.
 - The idea is to loop the outer loop 6 times and for each value, loop the inner loop 12 times. We use $n1$ as the outer loop control variable, and $n2$ for the inner loop's.

Example of Nested Loops

```
- for n1 = 1:6
-   for n2 = 1:12
-       fprintf('%4d', n1 * n2);
-   end
-   fprintf('\n');
end
```

Since they are counting loops, the for-end commands are recommended, but could you do the same with while-end commands instead?

```
>> mtable
 1   2   3   4   5   6   7   8   9  10  11  12
 2   4   6   8  10  12  14  16  18  20  22  24
 3   6   9  12  15  18  21  24  27  30  33  36
 4   8  12  16  20  24  28  32  36  40  44  48
 5  10  15  20  25  30  35  40  45  50  55  60
 6  12  18  24  30  36  42  48  54  60  66  72
>>
```

What about making a 12x12 multiplication table instead? Easy?

A Cool Use of a Loop

- Loops are an important programming construct, although in MATLAB we have seen that most loop operations can be done using one single elementwise command.
- Even the end-of-file controlled loop can be substituted as the `fscanf` command can read the entire file at once into a column vector. Once it is in the workspace, you can use the array commands and attributes (colon operator, ranges, indexes and subscripts).
- There are few things that require loops however like this next example when we use a loop to look around a three dimensional plot.

Looking Around a 3D Plot

```
c = [-pi : 0.1 : pi];  
d = [-pi : 0.1 : pi];  
[C,D] = meshgrid (c,d);  
E = sin (C) .* sin (D);  
mesh (C, D, E);
```

```
for n = -180:2:180  
    view (n,-n);  
    pause (.2)  
end
```

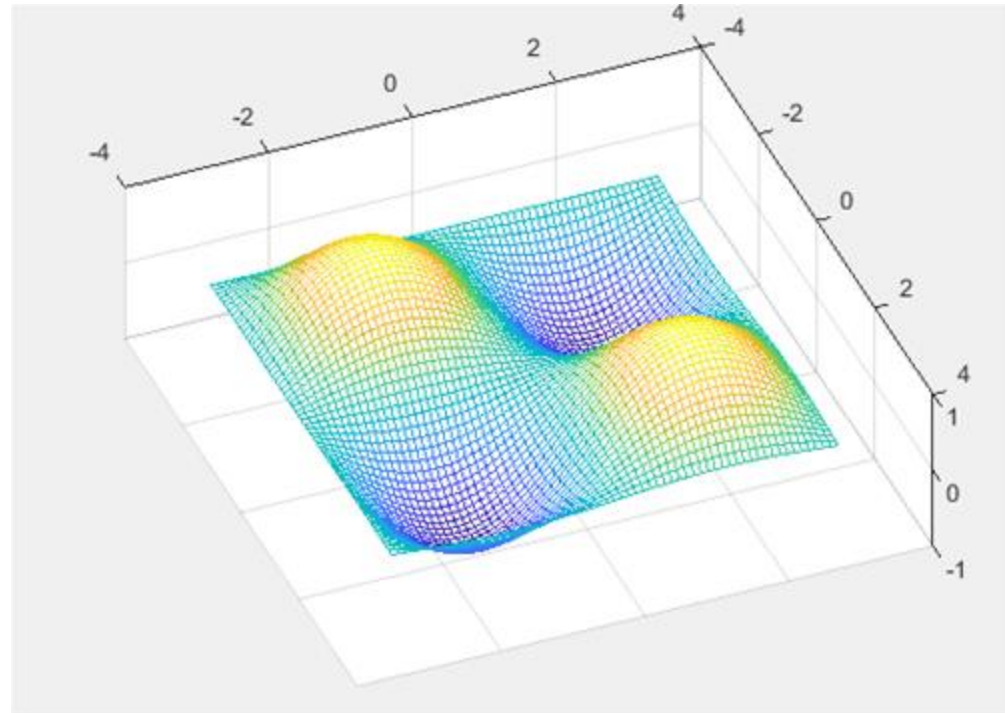
Looking Around a 3D Plot

```
c = [-pi : 0.1 : pi];  
d = [-pi : 0.1 : pi];  
[C,D] = meshgrid (c,d);  
E = sin (C) .* sin (D);  
mesh (C, D, E);
```

```
for n = -180:2:180  
    view (n,-n);  
    pause (.2)  
end
```

Awesome!

Try it in MATLAB and see
the animated result!



Lesson Overview

In this lesson we will study:

- **Polynomials:** Functions of a special form that arise often in science.
- **Curve Fitting:** Finding the exact function of a specified form that represents the data with the least amount of error.
- **Interpolation:** Estimating values between data points.

Polynomials

- MATLAB represents a polynomial by a row vector of coefficients.
 - The first vector element is the coefficient of x to highest power (c^n)
 - The second vector element is c^{n-1}
 - The n th vector element is c^1
 - The $n+1^{th}$ vector element is c^0

- MATLAB represents a polynomial of degree n by a vector of length $n+1$.
- By degree we mean the highest power of the independent variable appearing in the polynomial.
- For example, in the polynomial $6x^2 + 8x - 7$ the degree of the polynomial is 2.

Polynomial Coefficients

- The vector must include all the polynomial coefficients, even those that are zero.

Examples:

Polynomial

$$8x + 5$$

$$2x^2 - 4x + 10$$

$$6x^2 - 150, \text{ MATLAB form: } 6x^2 + 0x - 150$$

$$5x^5 + 6x^2 - 7x, \text{ MATLAB form:}$$

$$5x^5 + 0x^4 + 0x^3 + 6x^2 - 7x + 0$$

MATLAB representation

$$p = [8 \ 5]$$

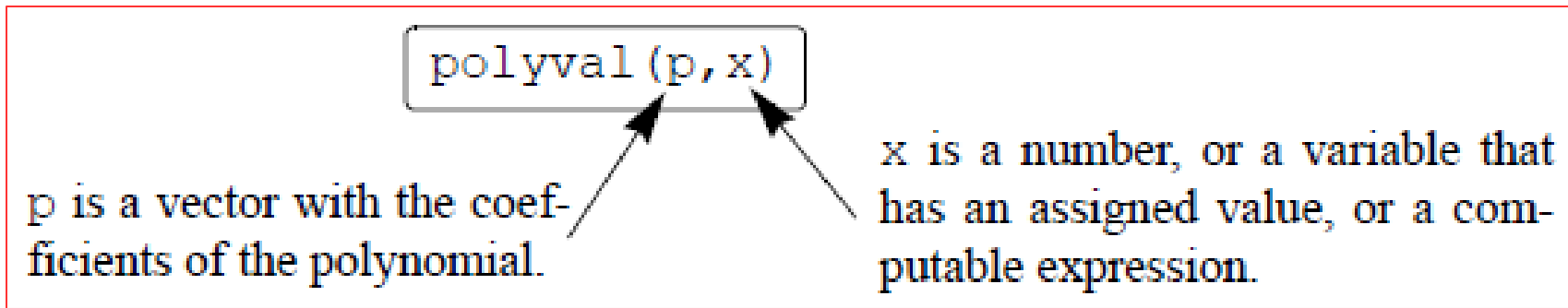
$$d = [2 \ -4 \ 10]$$

$$h = [6 \ 0 \ -150]$$

$$c = [5 \ 0 \ 0 \ 6 \ -7 \ 0]$$

Polynomials Values : The polyval Command

- The MATLAB command **polyval** computes the value of a polynomial at a point x (or at a series of points if x is a vector or a matrix).



- $\mathbf{p}$ is a vector of polynomial coefficients.
- $\mathbf{x}$ is a scalar, vector, or matrix of points at which the polynomial should be evaluated (done element-wise if x is a vector or a matrix).

○ The polyval Command : Example

```
a = [8 5]; b = polyval (a,3)
```

○ The polyval Command : Example

```
a = [8 5]; b = polyval (a,3)
```

```
b = 29
```

```
a = [8 5];
```

```
z = [1 3 7];
```

```
b = polyval (a, z)
```

○ The polyval Command : Example

```
a = [8 5]; b = polyval (a,3)
```

```
b = 29
```

```
a = [8 5];
```

```
z = [1 3 7];
```

```
b = polyval (a, z)
```

```
b = 13 29 61
```

Note: The second value is, again, 29

Roots of a Polynomial

- The *roots* of a polynomial are the values of the independent variable that make the polynomial to be zero.
- A polynomial of degree n has exactly n roots, though some may be repeated.
- The MATLAB command **roots** finds the roots of a polynomial.

```
r = roots(p)
```

r is a column vector with the roots of the polynomial.

p is a row vector with the coefficients of the polynomial.

The roots Command : Example

- Consider a polynomial **p** expressed as a vector of coefficients.

p = [1 -3 -3 7 6]

- Get the roots of the polynomial into a vertical vector **rts** with the roots command.

rts = roots(**p**)

rts =

3.0000

2.0000

-1.0000

-1.0000

Roots of a Polynomial and the **poly** Command

- If, on the other hand, you know the roots of a polynomial but not the coefficients, you can get them with the **poly** command.
- Let's try that command with our vector of roots **rts**.

```
coeffs = poly(rts)
```

Roots of a Polynomial and the poly Command

- If, on the other hand, you know the roots of a polynomial but not the coefficients, you can get them with the **poly** command.
- Let's try that command with our vector of roots **rts**.

```
coeffs = poly(rts)
```

```
coeffs =
```

```
      1.0000    -3.0000    -3.0000  
7.0000      6.0000
```

- Not surprisingly, you can see that **coeffs** is the same vector as our original polynomial **p**.

○ Addition/Subtraction of Polynomials

- To add (subtract) two polynomials, you add (subtract) their vectors of coefficients.
- If one vector is shorter than the other, you must add enough zeros in front of the shorter vector so that it gets to the same size as the longer vector.

$$5x^2 - 10x + 3$$

$$2x + 7$$

Addition/Subtraction of Polynomials

- To add (subtract) two polynomials, you add (subtract) their vectors of coefficients.
- If one vector is shorter than the other, you must add enough zeros in front of the shorter vector so that it gets to the same size as the longer vector.

```
>> p1 = [5 -10 3];
```

```
>> p2 = [0 2 7];
```

```
>> p3 = p1 + p2
```

```
p3 = 5 -8 10
```

$$5x^2 - 10x + 3$$

$$2x + 7$$

$$5x^2 - 8x + 10$$